

FDE PROGRAM

Techademy × EYGDS

Generative AI & Building Effective Agents

A Comprehensive Reference for EYGDS Learners

Sources: AWS — What is Generative AI? | Anthropic — Building Effective Agents

PART 1

What is Generative AI?

Source: Amazon Web Services (AWS)

What is Generative AI?

Generative artificial intelligence (generative AI) is a type of AI that can create new content and ideas, including conversations, stories, images, videos, and music. It can learn human language, programming languages, art, chemistry, biology, or any complex subject matter. It reuses what it knows to solve new problems. For example, it can learn English vocabulary and create a poem from the words it processes. Your organization can use generative AI for various purposes, like chatbots, media creation, product development, and design.

What is the Difference Between AI and Generative AI?

Artificial intelligence is the broader concept of making machines more human-like. It includes everything from smart assistants like Alexa, chatbots, and image generators to robotic vacuum cleaners and self-driving cars. Generative AI is a subset that generates new content meaningfully and intelligently.

When Was Generative AI Created?

Generative AI emerged in the late 2010s with advancements in deep learning, particularly with models like Generative Adversarial Networks (GANs) and transformers. Advances in cloud computing have made generative AI commercially viable and available since 2022.

What Are Foundation Models in Generative AI?

Foundation models are large generative AI models trained on a broad spectrum of text and image data. They are capable of performing a wide variety of general tasks like answering questions, writing essays, and captioning images.

What Are Generative AI Examples?

Generative AI has several use cases across industries.

Financial Services

Financial services companies use generative AI tools to serve their customers better while reducing costs:

- Financial institutions use chatbots to generate product recommendations and respond to customer inquiries, which improves overall customer service
- Lending institutions speed up loan approvals for financially underserved markets, especially in developing nations
- Banks quickly detect fraud in claims, credit cards, and loans
- Investment firms use the power of generative AI to provide safe, personalized financial advice to their clients at a low cost

Healthcare and Life Sciences

One of the most promising generative AI use cases is accelerating drug discovery and research. Generative AI can create novel protein sequences with specific properties for designing antibodies, enzymes, vaccines, and gene therapy.

Healthcare and life sciences companies use generative AI tools to design synthetic gene sequences for synthetic biology and metabolic engineering applications. For example, they can create new biosynthetic pathways or optimize gene expression for biomanufacturing purposes.

Generative AI tools also create synthetic patient and healthcare data. This data can be useful for training AI models, simulating clinical trials, or studying rare diseases without access to large real-world datasets.

Automotive and Manufacturing

Automotive companies use generative AI technology for many purposes, from engineering to in-vehicle experiences and customer service. For instance, they optimize the design of mechanical parts to reduce drag in vehicle designs or adapt the design of personal assistants.

Auto companies use generative AI tools to deliver better customer service by providing quick responses to the most common customer questions. Generative AI creates new materials, chips, and part designs to optimize manufacturing processes and reduce costs.

Another generative AI use case is synthesizing data to test applications. This is especially helpful for data not often included in testing datasets (such as defects or edge cases).

Media and Entertainment

From animations and scripts to full-length movies, generative AI models produce novel content at a fraction of the cost and time of traditional production.

Other generative AI use cases in the industry include:

- Artists can complement and enhance their albums with AI-generated music to create whole new experiences
- Media organizations use generative AI to improve their audience experiences by offering personalized content and ads to grow revenues
- Gaming companies use generative AI to create new games and allow players to build avatars

Telecommunication

Generative AI use cases in telecommunication focus on reinventing the customer experience defined by the cumulative interactions of subscribers across all touchpoints of the customer journey.

For instance, telecommunication organizations apply generative AI to improve customer service with live human-like conversational agents. They reinvent customer relationships with personalized one-to-one sales assistants. They also optimize network performance by analyzing network data to recommend fixes.

Energy

Generative AI suits energy sector tasks involving complex raw data analysis, pattern recognition, forecasting, and optimization. Energy organizations improve customer service by analyzing enterprise data to identify usage patterns. With this information, they can develop targeted product offerings, energy efficiency programs, or demand-response initiatives.

Generative AI also helps with grid management, increases operational site safety, and optimizes energy production through reservoir simulation.

What Are the Benefits of Generative AI?

According to Goldman Sachs, generative AI could drive a 7 percent (or almost \$7 trillion) increase in global gross domestic product (GDP) and lift productivity growth by 1.5 percentage points over ten years.

Benefit	Description
---------	-------------

Accelerates Research	Generative AI algorithms can explore and analyze complex data in new ways, allowing researchers to discover new trends and patterns that may not be otherwise apparent. These algorithms can summarize content, outline multiple solution paths, brainstorm ideas, and create detailed documentation from research notes.
Enhances Customer Experience	Generative AI can respond naturally to human conversation and serve as a tool for customer service and personalization of customer workflows, using AI-powered chatbots, voice bots, and virtual assistants for first-contact resolution.
Optimizes Business Processes	With generative AI, your business can optimize business processes utilizing machine learning (ML) and AI applications across all lines of business including engineering, marketing, customer service, finance, and sales.
Boosts Employee Productivity	Generative AI models can augment employee workflows and act as efficient assistants for everyone in your organization — from generating code suggestions and reports, to creating marketing content and supporting creative tasks.

How Did Generative AI Technology Evolve?

Primitive generative models have been used for decades in statistics to aid in numerical data analysis. Neural networks and deep learning were recent precursors for modern generative AI. Variational autoencoders, developed in 2013, were the first deep generative models that could generate realistic images and speech.

VAEs

VAEs introduced the capability to create novel variations of multiple data types. This led to the rapid emergence of other generative AI models like generative adversarial networks and diffusion models. These innovations were focused on generating data that increasingly resembled real data despite being artificially created.

Transformers

In 2017, a further shift in AI research occurred with the introduction of transformers. Transformers seamlessly integrated the encoder-and-decoder architecture with an attention mechanism. They streamlined the training process of language models with exceptional efficiency and versatility. Notable models like GPT emerged as foundational models capable of pretraining on extensive corpora of raw text and fine-tuning for diverse tasks.

Transformers changed what was possible for natural language processing. They empowered generative capabilities for tasks ranging from translation and summarization to answering questions.

The Future

Many generative AI models continue to make significant strides and have found cross-industry applications. Recent innovations focus on refining models to work with proprietary data. Researchers also want to create text, images, videos, and speech that are more and more human-like.

How Does Generative AI Work?

Like all artificial intelligence, generative AI works by using machine learning models—very large models that are pre-trained on vast amounts of data.

Foundation Models

Foundation models (FMs) are ML models trained on a broad spectrum of generalized and unlabeled data. They are capable of performing a wide variety of general tasks.

FMs are the result of the latest advancements in a technology that has been evolving for decades. In general, an FM uses learned patterns and relationships to predict the next item in a sequence.

For example, with image generation, the model analyzes the image and creates a sharper, more clearly defined version of the image. Similarly, with text, the model predicts the next word in a string of text based on the previous words and their context. It then selects the next word using probability distribution techniques.

Large Language Models

Large language models (LLMs) are one class of FMs. For example, OpenAI's generative pre-trained transformer (GPT) models are LLMs. LLMs are specifically focused on language-based tasks such as summarization, text generation, classification, open-ended conversation, and information extraction.

What makes LLMs special is their ability to perform multiple tasks. They can do this because they contain many parameters that make them capable of learning advanced concepts.

An LLM like GPT-3 can consider billions of parameters and has the ability to generate content from very little input. Through their pretraining exposure to internet-scale data in all its various forms and myriad patterns, LLMs learn to apply their knowledge in a wide range of contexts.

How Do Generative AI Models Work?

Traditional machine learning models were discriminative or focused on classifying data points. They attempted to determine the relationship between known and unknown factors. For example, they look at images—known data like pixel arrangement, line, color, and shape—and map them to words—the unknown factor. Mathematically, the models worked by identifying equations that could numerically map unknown and known factors as x and y variables.

Generative models take this one step further. Instead of predicting a label given some features, they try to predict features given a certain label. Mathematically, generative modeling calculates the probability of x and y occurring together. It learns the distribution of different data features and their relationships.

For example, generative models analyze animal images to record variables like different ear shapes, eye shapes, tail features, and skin patterns. They learn features and their relations to understand what different animals look like in general. They can then recreate new animal images that were not in the training set.

Below are some broad categories of generative AI models:

Diffusion Models

Diffusion models create new data by iteratively making controlled random changes to an initial data sample. They start with the original data and add subtle changes (noise), progressively making it less similar to the original. This noise is carefully controlled to ensure the generated data remains coherent and realistic.

After adding noise over several iterations, the diffusion model reverses the process. Reverse denoising gradually removes the noise to produce a new data sample that resembles the original.

Generative Adversarial Networks

The generative adversarial network (GAN) is another generative AI model that builds upon the diffusion model's concept.

GANs work by training two neural networks in a competitive manner. The first network, known as the generator, generates fake data samples by adding random noise. The second network called the discriminator, tries to distinguish between real data and the fake data produced by the generator.

During training, the generator continually improves its ability to create realistic data while the discriminator becomes better at telling real from fake. This adversarial process continues until the generator produces data that is so convincing that the discriminator can't differentiate it from real data.

GANs are widely used in generating realistic images, style transfer, and data augmentation tasks.

Variational Autoencoders

Variational autoencoders (VAEs) learn a compact representation of data called latent space. The latent space is a mathematical representation of the data. You can think of it as a unique code representing the data based on all its attributes. For example, if studying faces, the latent space contains numbers representing eye shape, nose shape, cheekbones, and ears.

VAEs use two neural networks—the encoder and the decoder. The encoder neural network maps the input data to a mean and variance for each dimension of the latent space. It generates a random sample from a Gaussian (normal) distribution. This sample is a point in the latent space and represents a compressed, simplified version of the input data.

The decoder neural network takes this sampled point from the latent space and reconstructs it back into data that resembles the original input. Mathematical functions are used to measure how well the reconstructed data matches the original data.

Transformer-Based Models

The transformer-based generative AI model builds upon the encoder and decoder concepts of VAEs. Transformer-based models add more layers to the encoder to improve performance on text-based tasks like comprehension, translation, and creative writing.

Transformer-based models use a self-attention mechanism. They weigh the importance of different parts of an input sequence when processing each element in the sequence.

Another key feature is that these AI models implement contextual embeddings. The encoding of a sequence element depends not only on the element itself but also on its context within the sequence.

To understand how transformer-based models work, imagine a sentence as a sequence of words. Self-attention helps the model focus on the relevant words as it processes each word. The transformer-based generative model employs multiple encoder layers called attention heads to capture different types of relationships between words. Each head learns to attend to different parts of the input sequence, allowing the model to simultaneously consider various aspects of the data.

Each layer also refines the contextual embeddings, making them more informative and capturing everything from grammar syntax to complex semantic meanings.

What Are the Limitations of Generative AI?

Despite their advancements, generative AI systems can sometimes produce inaccurate or misleading information. They rely on patterns and data they were trained on and can reflect biases or inaccuracies inherent in that data. Other concerns related to training data include:

Security

Data privacy and security concerns arise if proprietary data is used to customize generative AI models. Efforts must be made to ensure that the generative AI tools generate responses that limit unauthorized access to proprietary data. Security concerns also arise if there is a lack of accountability and transparency in how AI models make decisions.

Creativity

While generative AI can produce creative content, it often lacks true originality. The creativity of AI is bounded by the data it has been trained on, leading to outputs that may feel repetitive or derivative. Human creativity, which involves a deeper understanding and emotional resonance, remains challenging for AI to replicate fully.

Cost

Training and running generative AI models require substantial computational resources. Cloud-based generative AI models are more accessible and affordable than trying to build new models from scratch.

Explainability

Due to their complex and opaque nature, generative AI models are often considered black boxes. Understanding how these models arrive at specific outputs is challenging. Improving interpretability and transparency is essential to increase trust and adoption.

What Are the Best Practices in Generative AI Adoption?

If your organization wants to implement generative AI solutions, consider the following best practices to enhance your efforts.

Begin with Internal Applications

It's best to start generative AI adoption with internal application development, focusing on process optimization and employee productivity. You get a more controlled environment to test outcomes while building skills and understanding of the technology. You can test the models extensively and even customize them on internal knowledge sources.

This way, your customers have a much better experience when you eventually use the models for external applications.

Enhance Transparency

Clearly communicate about all generative AI applications and outputs so your users know they are interacting with AI and not humans. For instance, the AI can introduce itself as AI, or AI-based search results can be marked and highlighted.

That way, your users can use their own discretion when they engage with the content. They may also be more proactive in dealing with any inaccuracies or hidden biases the underlying models may have because of their training data limitations.

Implement Security

Implement guardrails so your generative AI applications don't allow inadvertent unauthorized access to sensitive data. Involve security teams from the start so all aspects can be considered from the beginning. For example, you may have to mask data and remove personally identifiable information (PII) before you train any models on internal data.

Test Extensively

Develop automated and manual testing processes to validate results and test all types of scenarios the generative AI system may experience. Have different groups of beta testers who try out the applications in different ways and document results. The model will also improve continuously through testing, and you get more control over expected outcomes and responses.

What is Generative AI Training for Beginners?

Generative AI training begins with understanding foundational machine learning concepts. Learners also have to explore neural networks and AI architecture. Practical experience with Python libraries such as TensorFlow or PyTorch is essential for implementing and experimenting with different models. You also have to learn model evaluation, fine tuning and prompt engineering skills.

A degree in artificial intelligence or machine learning provides in-depth training. Consider online short courses and certifications for professional development. Generative AI training includes certifications by experts on topics like:

- Introduction to generative AI
- Generative AI for executives
- Generative AI essentials for business

How Can AWS Help with Generative AI?

Amazon Web Services (AWS) makes it easy to build and scale generative AI applications for your data, use cases, and customers. With generative AI on AWS, you get enterprise-grade security and privacy, access to industry-leading FMs, generative AI-powered applications, and a data-first approach.

AWS offers a range of generative AI services that support all types of organizations in every stage of generative AI adoption and maturity:

- Amazon Q Developer — a generative AI-powered assistant for software development that delivers great results in developer productivity
- Amazon Bedrock — a fully managed service that offers a choice of high-performing FMs and a broad set of capabilities for experimenting, customizing, and creating managed agents
- Amazon SageMaker JumpStart — enables discovering, exploring, and deploying open source FMs with managed infrastructure and tools to accelerate scalable, reliable, and secure model building
- AWS HealthScribe — a HIPAA-eligible service empowering healthcare software vendors to build clinical applications that automatically generate clinical notes by analyzing patient-clinician conversations
- Amazon Q in QuickSight — helps business analysts easily create and customize visuals using natural-language commands

PART 2

Building Effective Agents

Source: Anthropic Engineering Blog | Published: December 19, 2024

We've worked with dozens of teams building LLM agents across industries. Consistently, the most successful implementations use simple, composable patterns rather than complex frameworks.

Over the past year, we've worked with dozens of teams building large language model (LLM) agents across industries. Consistently, the most successful implementations weren't using complex frameworks or specialized libraries. Instead, they were building with simple, composable patterns.

In this post, we share what we've learned from working with our customers and building agents ourselves, and give practical advice for developers on building effective agents.

What Are Agents?

"Agent" can be defined in several ways. Some customers define agents as fully autonomous systems that operate independently over extended periods, using various tools to accomplish complex tasks. Others use the term to describe more prescriptive implementations that follow predefined workflows. At Anthropic, we categorize all these variations as agentic systems, but draw an important architectural distinction between workflows and agents:

Workflows	Systems where LLMs and tools are orchestrated through predefined code paths.
Agents	Systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.

Below, we will explore both types of agentic systems in detail.

When (and When Not) to Use Agents

When building applications with LLMs, we recommend finding the simplest solution possible, and only increasing complexity when needed. This might mean not building agentic systems at all. Agentic systems often trade latency and cost for better task performance, and you should consider when this tradeoff makes sense.

When more complexity is warranted, workflows offer predictability and consistency for well-defined tasks, whereas agents are the better option when flexibility and model-driven decision-making are needed at scale. For many applications, however, optimizing single LLM calls with retrieval and in-context examples is usually enough.

When and How to Use Frameworks

There are many frameworks that make agentic systems easier to implement, including:

- The Claude Agent SDK
- Strands Agents SDK by AWS
- Rivet, a drag and drop GUI LLM workflow builder
- Vellum, another GUI tool for building and testing complex workflows

These frameworks make it easy to get started by simplifying standard low-level tasks like calling LLMs, defining and parsing tools, and chaining calls together. However, they often create extra layers of

abstraction that can obscure the underlying prompts and responses, making them harder to debug. They can also make it tempting to add complexity when a simpler setup would suffice.

We suggest that developers start by using LLM APIs directly: many patterns can be implemented in a few lines of code. If you do use a framework, ensure you understand the underlying code. Incorrect assumptions about what's under the hood are a common source of customer error.

Building Blocks, Workflows, and Agents

In this section, we'll explore the common patterns for agentic systems we've seen in production. We'll start with our foundational building block—the augmented LLM—and progressively increase complexity, from simple compositional workflows to autonomous agents.

Building Block: The Augmented LLM

The basic building block of agentic systems is an LLM enhanced with augmentations such as retrieval, tools, and memory. Our current models can actively use these capabilities—generating their own search queries, selecting appropriate tools, and determining what information to retain.

We recommend focusing on two key aspects of the implementation: tailoring these capabilities to your specific use case and ensuring they provide an easy, well-documented interface for your LLM. While there are many ways to implement these augmentations, one approach is through the Model Context Protocol, which allows developers to integrate with a growing ecosystem of third-party tools with a simple client implementation.

For the remainder of this post, we'll assume each LLM call has access to these augmented capabilities.

Workflow: Prompt Chaining

Prompt chaining decomposes a task into a sequence of steps, where each LLM call processes the output of the previous one. You can add programmatic checks on any intermediate steps to ensure that the process is still on track.

When to use

This workflow is ideal for situations where the task can be easily and cleanly decomposed into fixed subtasks. The main goal is to trade off latency for higher accuracy, by making each LLM call an easier task.

Examples where prompt chaining is useful:

- Generating Marketing copy, then translating it into a different language
- Writing an outline of a document, checking that the outline meets certain criteria, then writing the document based on the outline

Workflow: Routing

Routing classifies an input and directs it to a specialized followup task. This workflow allows for separation of concerns, and building more specialized prompts. Without this workflow, optimizing for one kind of input can hurt performance on other inputs.

When to use

Routing works well for complex tasks where there are distinct categories that are better handled separately, and where classification can be handled accurately, either by an LLM or a more traditional classification model/algorithm.

Examples where routing is useful:

- Directing different types of customer service queries (general questions, refund requests, technical support) into different downstream processes, prompts, and tools
- Routing easy/common questions to smaller, cost-efficient models like Claude Haiku 4.5 and hard/unusual questions to more capable models like Claude Sonnet 4.5 to optimize for best performance

Workflow: Parallelization

LLMs can sometimes work simultaneously on a task and have their outputs aggregated programmatically. This workflow, parallelization, manifests in two key variations:

- Sectioning: Breaking a task into independent subtasks run in parallel
- Voting: Running the same task multiple times to get diverse outputs

When to use

Parallelization is effective when the divided subtasks can be parallelized for speed, or when multiple perspectives or attempts are needed for higher confidence results. For complex tasks with multiple considerations, LLMs generally perform better when each consideration is handled by a separate LLM call.

Examples where parallelization is useful:

Sectioning:

- Implementing guardrails where one model instance processes user queries while another screens them for inappropriate content or requests
- Automating evals for evaluating LLM performance, where each LLM call evaluates a different aspect of the model's performance on a given prompt

Voting:

- Reviewing a piece of code for vulnerabilities, where several different prompts review and flag the code if they find a problem
- Evaluating whether a given piece of content is inappropriate, with multiple prompts evaluating different aspects or requiring different vote thresholds to balance false positives and negatives

Workflow: Orchestrator-Workers

In the orchestrator-workers workflow, a central LLM dynamically breaks down tasks, delegates them to worker LLMs, and synthesizes their results.

When to use

This workflow is well-suited for complex tasks where you can't predict the subtasks needed (in coding, for example, the number of files that need to be changed and the nature of the change in each file likely depend on the task). The key difference from parallelization is its flexibility—subtasks aren't pre-defined, but determined by the orchestrator based on the specific input.

Example where orchestrator-workers is useful:

- Coding products that make complex changes to multiple files each time
- Search tasks that involve gathering and analyzing information from multiple sources for possible relevant information

Workflow: Evaluator-Optimizer

In the evaluator-optimizer workflow, one LLM call generates a response while another provides evaluation and feedback in a loop.

When to use

This workflow is particularly effective when we have clear evaluation criteria, and when iterative refinement provides measurable value. The two signs of good fit are: first, that LLM responses can be demonstrably improved when a human articulates their feedback; and second, that the LLM can provide such feedback.

Examples where evaluator-optimizer is useful:

- Literary translation where there are nuances that the translator LLM might not capture initially, but where an evaluator LLM can provide useful critiques
- Complex search tasks that require multiple rounds of searching and analysis to gather comprehensive information, where the evaluator decides whether further searches are warranted

Agents

Agents are emerging in production as LLMs mature in key capabilities—understanding complex inputs, engaging in reasoning and planning, using tools reliably, and recovering from errors. Agents begin their work with either a command from, or interactive discussion with, the human user. Once the task is clear, agents plan and operate independently, potentially returning to the human for further information or judgement. During execution, it's crucial for the agents to gain "ground truth" from the environment at each step (such as tool call results or code execution) to assess its progress. Agents can then pause for human feedback at checkpoints or when encountering blockers. The task often terminates upon completion, but it's also common to include stopping conditions (such as a maximum number of iterations) to maintain control.

Agents can handle sophisticated tasks, but their implementation is often straightforward. They are typically just LLMs using tools based on environmental feedback in a loop. It is therefore crucial to design toolsets and their documentation clearly and thoughtfully.

When to use

Agents can be used for open-ended problems where it's difficult or impossible to predict the required number of steps, and where you can't hardcode a fixed path. The LLM will potentially operate for many turns, and you must have some level of trust in its decision-making. Agents' autonomy makes them ideal for scaling tasks in trusted environments.

The autonomous nature of agents means higher costs, and the potential for compounding errors. We recommend extensive testing in sandboxed environments, along with the appropriate guardrails.

Examples where agents are useful (from Anthropic's own implementations):

- A coding Agent to resolve SWE-bench tasks, which involve edits to many files based on a task description
- The "computer use" reference implementation, where Claude uses a computer to accomplish tasks

Combining and Customizing These Patterns

These building blocks aren't prescriptive. They're common patterns that developers can shape and combine to fit different use cases. The key to success, as with any LLM features, is measuring performance and iterating on implementations. To repeat: you should consider adding complexity only when it demonstrably improves outcomes.

Summary

Success in the LLM space isn't about building the most sophisticated system. It's about building the right system for your needs. Start with simple prompts, optimize them with comprehensive evaluation, and add multi-step agentic systems only when simpler solutions fall short.

When implementing agents, we try to follow three core principles:

- 1 Simplicity** — Maintain simplicity in your agent's design.
- 2 Transparency** — Prioritize transparency by explicitly showing the agent's planning steps.
- 3 Documentation & Testing** — Carefully craft your agent-computer interface (ACI) through thorough tool documentation and testing.

Frameworks can help you get started quickly, but don't hesitate to reduce abstraction layers and build with basic components as you move to production. By following these principles, you can create agents that are not only powerful but also reliable, maintainable, and trusted by their users.

Appendix 1: Agents in Practice

Our work with customers has revealed two particularly promising applications for AI agents that demonstrate the practical value of the patterns discussed above. Both applications illustrate how agents add the most value for tasks that require both conversation and action, have clear success criteria, enable feedback loops, and integrate meaningful human oversight.

A. Customer Support

Customer support combines familiar chatbot interfaces with enhanced capabilities through tool integration. This is a natural fit for more open-ended agents because:

- Support interactions naturally follow a conversation flow while requiring access to external information and actions
- Tools can be integrated to pull customer data, order history, and knowledge base articles
- Actions such as issuing refunds or updating tickets can be handled programmatically
- Success can be clearly measured through user-defined resolutions

Several companies have demonstrated the viability of this approach through usage-based pricing models that charge only for successful resolutions, showing confidence in their agents' effectiveness.

B. Coding Agents

The software development space has shown remarkable potential for LLM features, with capabilities evolving from code completion to autonomous problem-solving. Agents are particularly effective because:

- Code solutions are verifiable through automated tests
- Agents can iterate on solutions using test results as feedback
- The problem space is well-defined and structured
- Output quality can be measured objectively

In our own implementation, agents can now solve real GitHub issues in the SWE-bench Verified benchmark based on the pull request description alone. However, whereas automated testing helps verify functionality, human review remains crucial for ensuring solutions align with broader system requirements.

Appendix 2: Prompt Engineering Your Tools

No matter which agentic system you're building, tools will likely be an important part of your agent. Tools enable Claude to interact with external services and APIs by specifying their exact structure and definition in our API. When Claude responds, it will include a tool use block in the API response if it plans to invoke a tool. Tool definitions and specifications should be given just as much prompt engineering attention as your overall prompts.

There are often several ways to specify the same action. For instance, you can specify a file edit by writing a diff, or by rewriting the entire file. For structured output, you can return code inside markdown or inside JSON. In software engineering, differences like these are cosmetic and can be converted losslessly from one to the other. However, some formats are much more difficult for an LLM to write than others. Writing a diff requires knowing how many lines are changing in the chunk header before the new code is written. Writing code inside JSON (compared to markdown) requires extra escaping of newlines and quotes.

Our suggestions for deciding on tool formats are the following:

- Give the model enough tokens to "think" before it writes itself into a corner
- Keep the format close to what the model has seen naturally occurring in text on the internet

- Make sure there's no formatting "overhead" such as having to keep an accurate count of thousands of lines of code, or string-escaping any code it writes

One rule of thumb is to think about how much effort goes into human-computer interfaces (HCI), and plan to invest just as much effort in creating good agent-computer interfaces (ACI). Here are some thoughts on how to do so:

- Put yourself in the model's shoes. Is it obvious how to use this tool, based on the description and parameters, or would you need to think carefully about it? If so, then it's probably also true for the model. A good tool definition often includes example usage, edge cases, input format requirements, and clear boundaries from other tools.
- How can you change parameter names or descriptions to make things more obvious? Think of this as writing a great docstring for a junior developer on your team. This is especially important when using many similar tools.
- Test how the model uses your tools: Run many example inputs in our workbench to see what mistakes the model makes, and iterate.
- Poka-yoke your tools. Change the arguments so that it is harder to make mistakes.

While building our agent for SWE-bench, we actually spent more time optimizing our tools than the overall prompt. For example, we found that the model would make mistakes with tools using relative filepaths after the agent had moved out of the root directory. To fix this, we changed the tool to always require absolute filepaths—and we found that the model used this method flawlessly.

Acknowledgements

Written by Erik S. and Barry Zhang. This work draws upon our experiences building agents at Anthropic and the valuable insights shared by our customers, for which we're deeply grateful.